Academia de Studii Economice din București

Facultatea de Cibernetică, Statistică și Informatică Economică

PROIECT: REȚELE DE CALCULATOARE

Sistem Distribuit pentru Coordonarea și Execuția Interogărilor la Distanță

Gheoala Alexandru Iliuta

Coman Mihaela Adelina

Copaciu Anisia

1. Introducere

Proiectul constă în dezvoltarea unei aplicații distribuite de tip client-server, care permite executarea la distanță a unor interogări (tip WMI/comenzi de sistem) și colectarea agregată a rezultatelor.

Sistemul permite utilizatorilor să se conecteze concurent la un server coordonator. Odată conectați, aceștia primesc și mențin o listă actualizată a tuturor membrilor din rețea. Un client cu rol de „operator" poate trimite interogări fie către toată rețeaua, fie către un anumit dispozitiv țintă, iar serverul se ocupă de distribuirea comenzii și colectarea răspunsurilor.

Aplicația este construită folosind limbajul Python, comunicarea realizându-se prin intermediul socket-urilor de rețea pe protocolul TCP (Transmission Control Protocol). Pentru a asigura portabilitatea și izolarea mediului de execuție, componenta de server a fost containerizată folosind tehnologia Docker. Scopul principal al proiectului este demonstrarea conceptelor de concurență (multi-threading), coordonare distribuită, gestionarea stării clienților și tratarea timeout-urilor într-o rețea.

2. Arhitectura Aplicației

Aplicația respectă arhitectura clasică **Client-Server**:

- **Serverul:** Acționează ca un nod central și coordonator. El menține starea globală a sistemului (dicționarul cu clienții conectați și adresele lor IP) și propagă orice schimbare (conectări/deconectări) către restul rețelei. Totodată, primește comenzile de execuție de la operatori, le distribuie către țintele vizate, așteaptă rezultatele asincron și le returnează sub formă agregată.
- **Clientul:** Funcționează atât ca *Operator* (trimite comenzi), cât și ca *Țintă* (răspunde la comenzi). Preia datele de la tastatură, le formatează sub formă de mesaje JSON și ascultă permanent mesajele venite de la server într-un fir de execuție separat.

2.1. Tehnologii și Protocoale Utilizate

- **Protocolul de transport (TCP):** A fost ales TCP deoarece este critic ca mesajele (înregistrarea în rețea, rezultatele comenzilor) să ajungă fiabil și fără pierderi de pachete. Pentru a preveni fragmentarea pachetelor, mesajele sunt delimitate de caracterul newline (`\n`).
- **Formatul datelor (JSON):** Schimbul de mesaje structurate între client și server (ex. tipul comenzii, rezultatele executate) se face formatat în JSON, asigurând o parsare ușoară.
- **Multi-threading:** Folosit pe server pentru a deservi mai mulți clienți simultan și pe client pentru a putea citi input-ul utilizatorului și a afișa instantaneu alertele de la server.
- **Docker:** Folosit pentru a izola mediul serverului, expunând portul `8719` pentru comunicare.

3. Detalii de Implementare

3.1. Componenta Server (server.py)

Serverul ascultă pe interfața `0.0.0.0` pentru a accepta conexiuni din exteriorul containerului Docker.

- **Gestionarea Concurenței și Sincronizarea:** Fiecare conexiune este preluată de un thread separat. Deoarece resursele (lista de clienți, sarcinile în așteptare) sunt partajate, se folosește un mecanism de tip `threading.RLock()` pentru a preveni coliziunile de date.
- **Tratarea Interogărilor și a Timeout-urilor:** Când se primește o cerere `query`, serverul generează un `task_id` unic și utilizează `threading.Event().wait(TIMEOUT)` pentru a aștepta rezultatele. Dacă un client nu răspunde în timpul alocat (5 secunde), serverul marchează rezultatul cu o eroare de Timeout, prevenind astfel blocarea întregului proces.

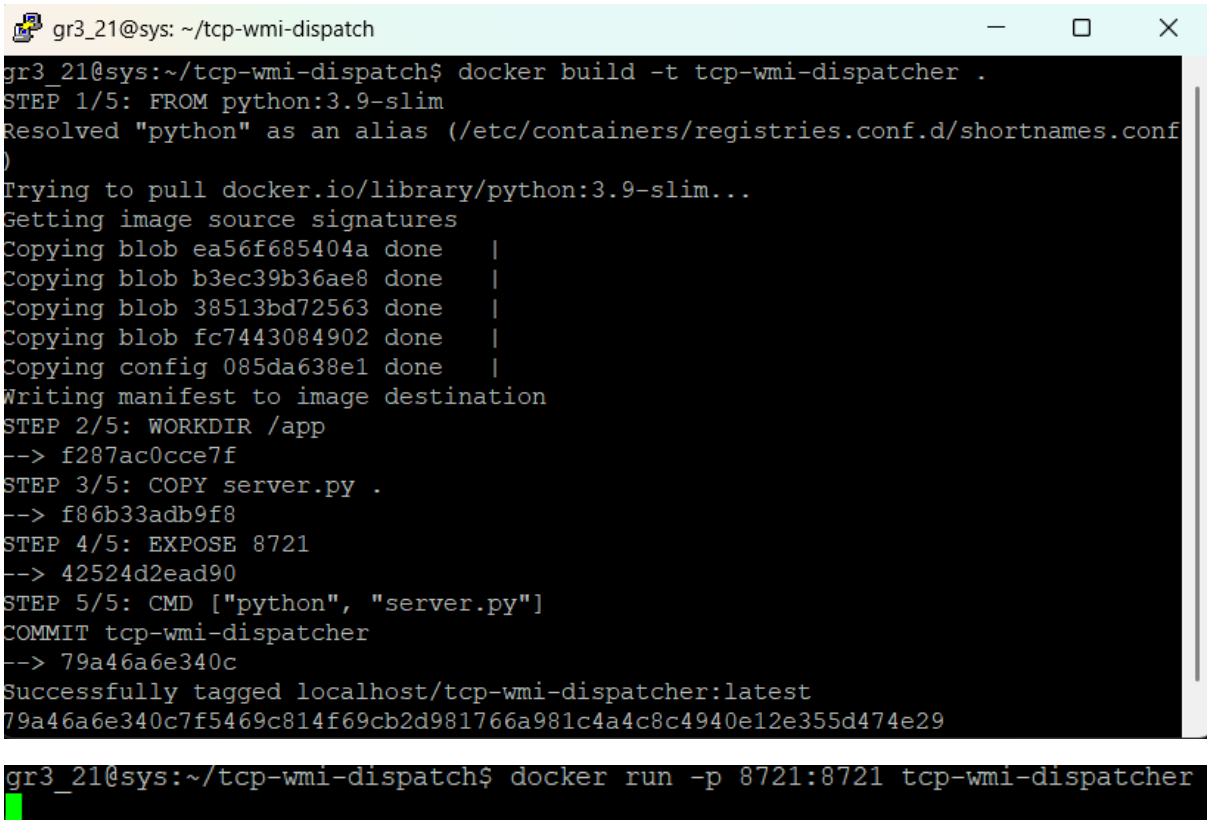
3.2. Componenta Client (client.py)

Clientul simulează funcționalitatea WMI printr-o serie de comenzi de sistem sigure (`os`, `time`, `ping`), extrăgând date despre mașina locală.

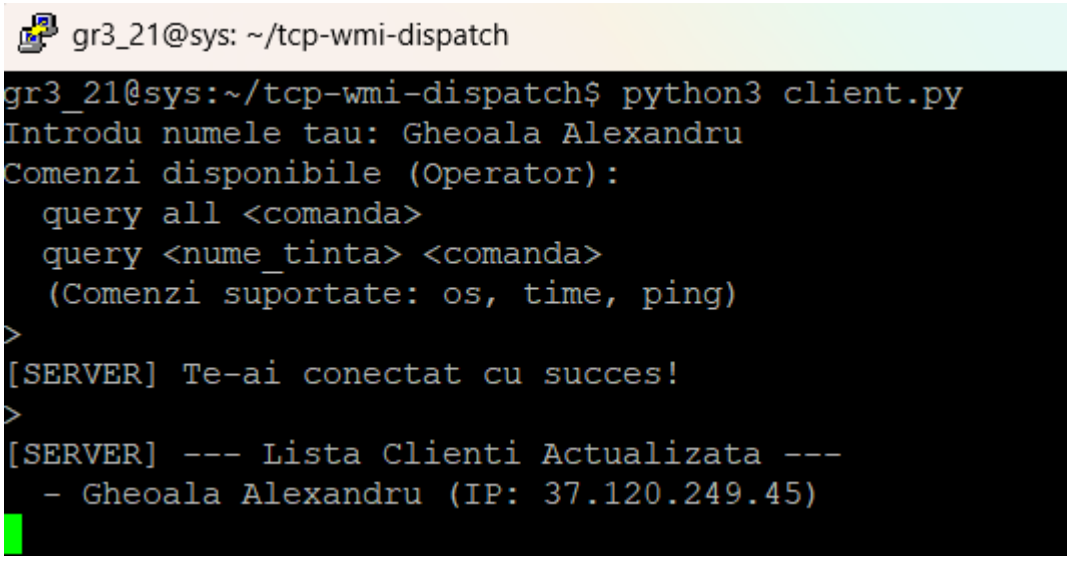
- **Thread-ul principal:** Procesează comenzile de tip operator (`query all <comanda>` sau `query <tinta> <comanda>`).
- **Thread-ul daemon (`asculta_server`):** Rulează în fundal și procesează instantaneu evenimentele de tip broadcast (actualizarea listei de clienți) sau solicitările de execuție venite de la coordonator.

4. Utilizarea Aplicației

Pornirea serverului:



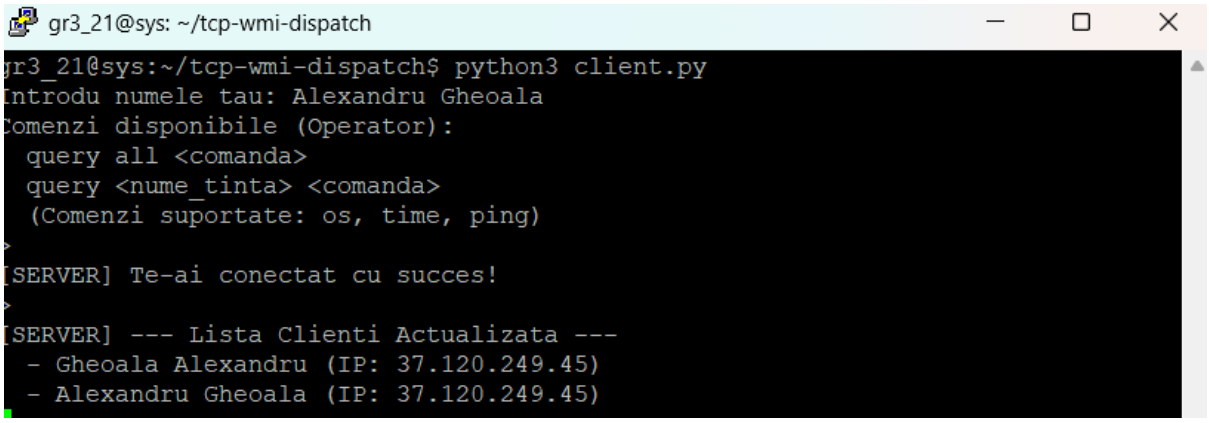
Conectarea primului client:

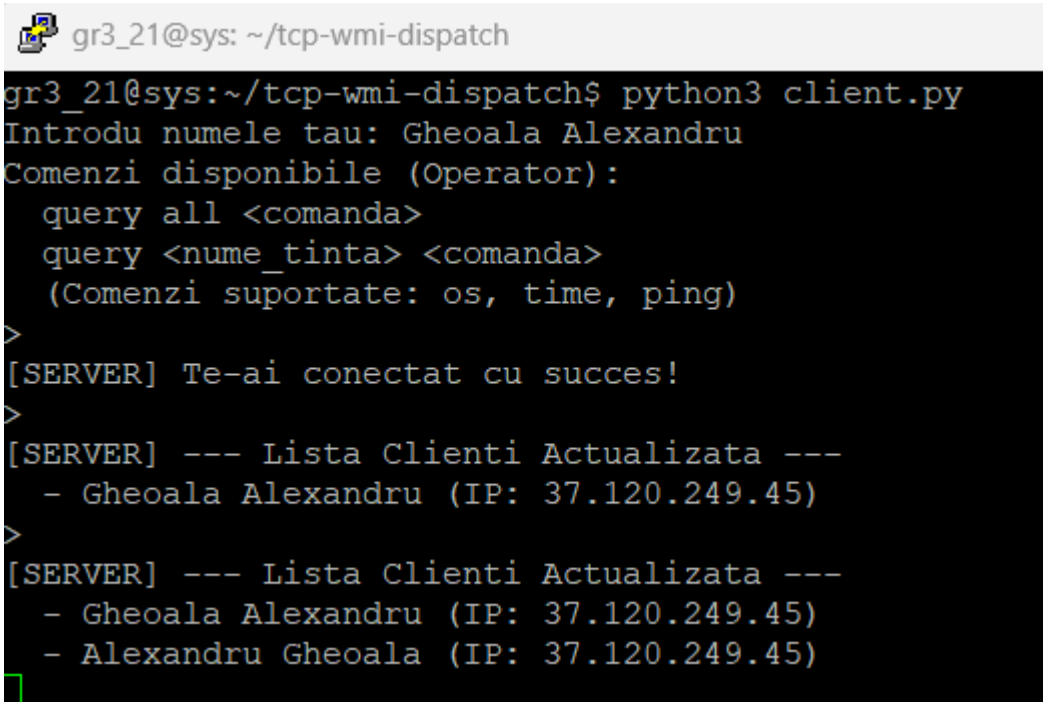


Verificare existență nume:



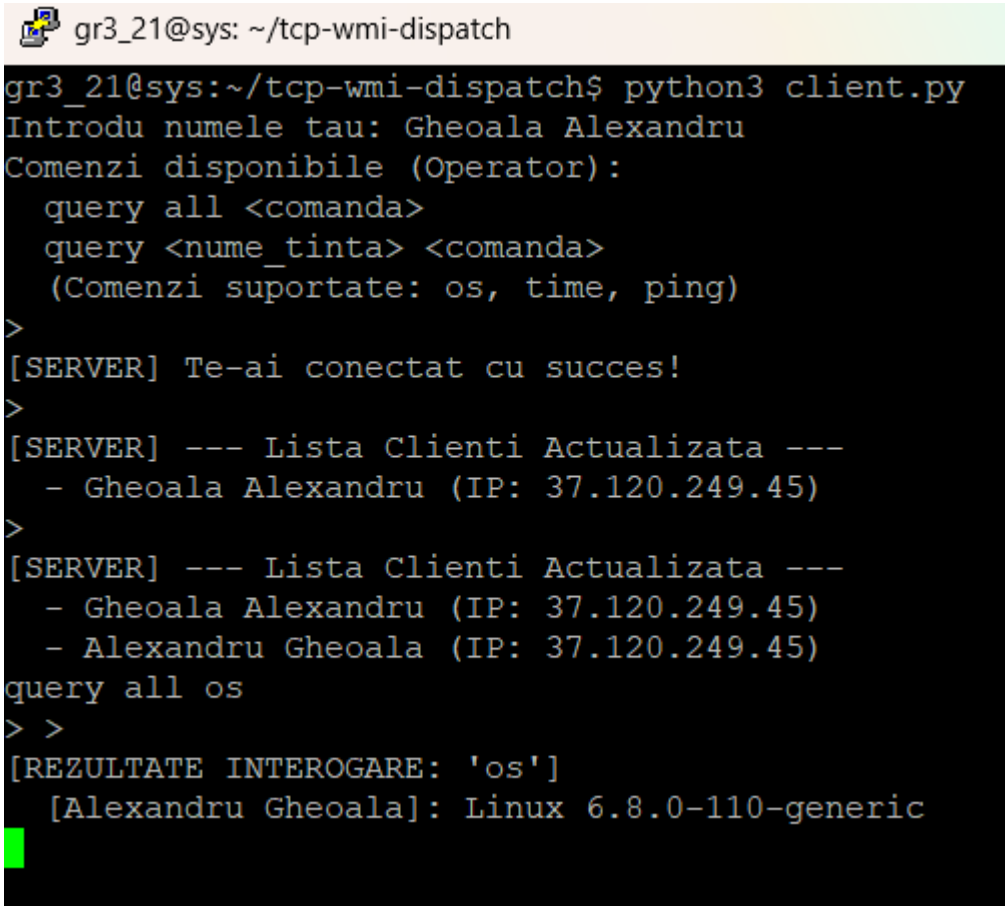
Notificarea clienților existenți la conectarea unei noi ținte:



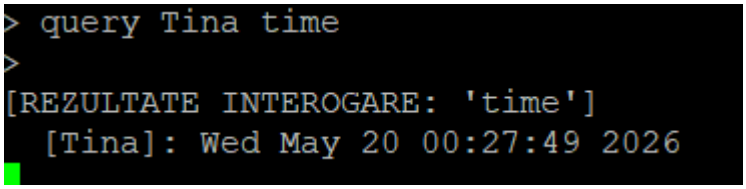


Comenzi disponibile pentru operatori: Sintaxa: `query all <comanda>` sau `query <tinta> <comanda>`

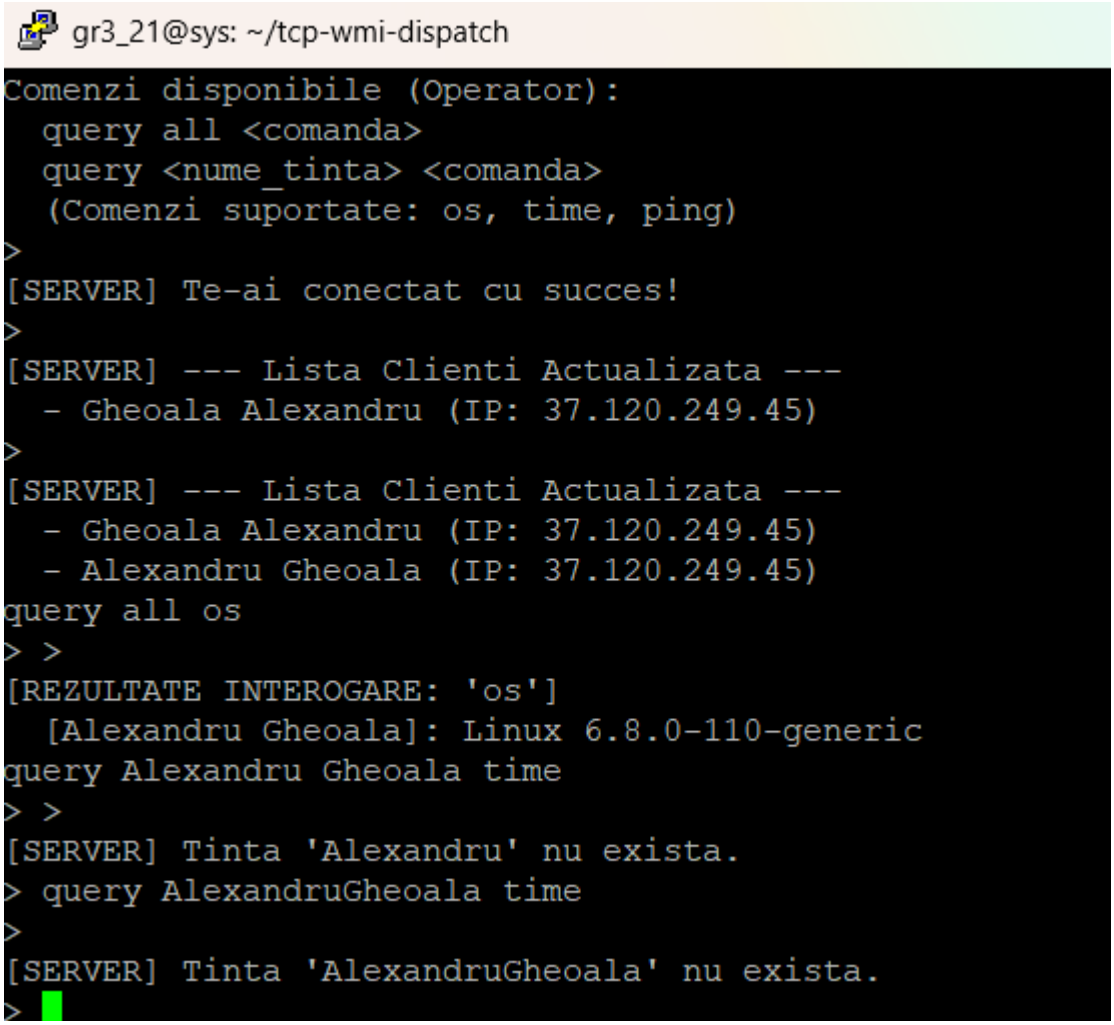
1. Interogare trimisă către toți clienții din rețea:



2. Interogare trimisă către o țintă specifică:



3. Tratarea introducerii unei ținte inexistente:



Deconectarea neașteptată a unui client: Când un client pierde conexiunea sau se deconectează, serverul propagă imediat modificarea către restul rețelei.